



Mediacrest Training College

Nairobi, Kenya

Advanced Data Analytics & Machine Learning

8-Week Advanced Training Program

Comprehensive Session Notes

Week 2 · Day 1

Outlier Detection, Statistical Thinking & Data Normalization in Financial Records

Course Trainer: Victor Wandera Lumumba

Role: Statistician, Data Analyst & BI Specialist

Session Date: 18 August 2026

Mode: Hybrid – Online (6–8 PM) / Physical (9–11 AM EAT)

“Understand the data, identify the problem, select the transformation, justify the decision.”

About This Document

These comprehensive notes accompany **Week 2, Day 1** of the **Advanced Data Analytics & Machine Learning** program. In Week 1, we established the foundational principles of data auditing, missingness mechanisms (MCAR, MAR, MNAR), and clinical imputation strategies using a Ministry of Health (MOH) dataset.

Today, we transition from the healthcare sector to the **financial sector**, applying our rigorous preprocessing mindset to a **Kenyan SACCO and Bank Churn Dataset** comprising 10,000 customer records. The core philosophy of this session is **transferability**: the dataset may change, the domain may change, but the underlying statistical logic and preprocessing workflows remain universally applicable. Whether you are analyzing patient vitals, bank account balances, or agricultural yields, the sequence of *Inspect* → *Identify* → *Clean* → *Transform* → *Encode* → *Validate* → *Model* is your invariant blueprint for success.

Course at a Glance

	8 Weeks (20+ intensive sessions)
Duration	
Core Tools	Python (pandas, NumPy, SciPy, scikit-learn, statsmodels), Power BI
Focus	Deep statistical modelling, supervised ML, feature engineering, algorithm alignment
Capstone	Production-ready ML system with diagnostics, explainability & policy recommendations
Kenyan Context	Healthcare (MOH), Finance (SACCOs/Banks), agriculture, pub- lic policy (CBK, KRA)

Where This Session Fits

Week 2 is titled “**Advanced Feature Engineering, Dimensionality Reduction & Model Alignment.**” It builds directly upon the clean, imputed data we produced in Week 1.

- Day 1 (this document):** Outlier Detection & Treatment, Feature Encoding Strategies (Ordinal, One-Hot, K-Fold Target), Scaling, and Algorithm Sensitivity.
- Day 2:** Feature Selection techniques (Filter, Wrapper, Embedded) and handling multi-collinearity.
- Day 3:** Dimensionality Reduction (PCA, t-SNE) and building the first predictive baselines.

How to Use These Notes

- Follow along in code.** Every listing is runnable. Open the companion Jupyter Notebook (*Week-2-Day-1-Outlier Detection-Handling-Statistical thinking and Data Normalization.ipynb*) and execute each cell as you read.

- **Boxes matter.** **Key Concept** boxes hold the theory; **Warning** boxes hold the mistakes that fail models; **Kenyan Context** boxes connect the material to local industry realities; **Mathematical Foundation** boxes provide the rigorous statistical proofs.
- **Do the exercises.** Each chapter ends with practice tasks designed to test your statistical intuition, not just your coding syntax.

Contents

About This Document	1
1 The Transferable Preprocessing Mindset	4
1.1 From Healthcare to Finance: The Universal Workflow	4
1.2 Key Characteristics of Financial Data	4
2 Data Loading & Initial Profiling	6
2.1 Loading the Dataset	6
2.2 Inspecting Data Types and Memory	6
3 Outlier Detection: Visual & Statistical Methods	8
3.1 Visual Detection: The Boxplot	8
3.2 Statistical Detection: The IQR Method (Tukey's Fences)	8
3.3 Statistical Detection: The Z-Score Method	9
3.4 The Discrepancy: Why IQR and Z-Score Disagree	10
4 Outlier Treatment Decisions: The Business Context	11
4.1 The Golden Rule: Outlier $\neq$ Error	11
4.2 Treatment Strategy: Capping (Winsorization)	11
4.3 Advantages of Capping over Deletion	12
5 Feature Encoding Strategies	13
5.1 Strategy 1: Ordinal Encoding (Preserving Hierarchy)	13
5.2 Strategy 2: One-Hot Encoding (Low Cardinality)	13
5.3 Strategy 3: Target Encoding (High Cardinality)	13
5.3.1 The Danger of Target Leakage	14
5.3.2 The Solution: K-Fold Target Encoding	14
6 Scaling, Normalization & Algorithm Sensitivity	16
6.1 Why Scale Features?	16
6.2 Comparing Scalers: Standard vs MinMax vs Robust	16
6.3 The MinMax Trap and The Robust Solution	17
6.4 Statistical Clarification: The Normality Myth	17
7 Practical Application & Final Pipeline	18
7.1 Selecting the Final Features	18
7.2 The Preprocessing Memo (Deliverable)	18
8 Best Practices Checklist	19
9 Glossary of Key Terms	20
10 References & Further Reading	20
11 Session Deliverables & What Comes Next	21

1 The Transferable Preprocessing Mindset

1.1 From Healthcare to Finance: The Universal Workflow

In data science, it is easy to become overly attached to a specific dataset or domain. However, the mark of a senior data scientist is the ability to abstract the *statistical structure* of a problem away from its *business context*.

In Week 1, we dealt with missing hemoglobin levels and skewed creatinine labs. Today, we deal with skewed account balances and high-cardinality geographical data. The variables have different names, but their mathematical behaviors are identical. A heavily right-skewed financial balance requires the exact same robust scaling technique as a heavily right-skewed clinical lab result.

The Universal Preprocessing Workflow

Regardless of the industry, the preprocessing pipeline follows a strict, invariant sequence:

1. **Inspect:** Profile distributions, data types, and memory usage.
2. **Identify:** Flag missingness, outliers, skewness, and cardinality issues.
3. **Clean:** Handle errors, impossible values, and duplicates.
4. **Transform:** Address skewness, cap extremes, and scale features.
5. **Encode:** Convert categoricals to numerical formats without injecting false math.
6. **Validate:** Ensure no data leakage or information loss has occurred.
7. **Model:** Feed model-ready, algorithm-aligned data to the estimator.

1.2 Key Characteristics of Financial Data

Financial datasets, particularly those from Kenyan retail banks, microfinance institutions, and SACCOs (Savings and Credit Cooperative Organizations), present unique statistical challenges that differ from clinical or survey data.

- **Skewed Numerical Variables:** Financial variables such as account balances, transaction volumes, and income almost universally exhibit a long right tail (positive skew). A small number of high-net-worth individuals or institutional SMEs hold a disproportionate amount of the total assets. Standard statistical measures (mean, standard deviation) are heavily distorted by this skew.
- **High-Cardinality Categorical Variables:** Variables representing geography (e.g., Kenya's 47 Counties, or hundreds of specific bank branches) or specific product codes contain dozens or hundreds of unique categories. Naively applying standard encoding techniques will result in the "Curse of Dimensionality."
- **Data Types and Consistency:** Financial systems often export identifiers (like Customer ID or Branch Code) as integers. If not explicitly cast to strings or categorical types, machine learning algorithms may mistakenly treat these IDs as continuous numerical features, inventing false mathematical relationships (e.g., assuming Branch 10 is "greater" than Branch 2).

The SACCO vs. Commercial Bank Context

In Kenya, SACCOs operate differently from tier-1 commercial banks. SACCO members often share dividends, take out multi-purpose loans, and exhibit highly seasonal deposit behaviors tied to agricultural harvest cycles or end-of-year bonuses. When preprocessing SACCO data, an “outlier” balance in December might be a legitimate, expected seasonal spike rather than a data-entry error. Domain knowledge is your ultimate preprocessing guide.

2 Data Loading & Initial Profiling

2.1 Loading the Dataset

The raw data file (`churn_data.csv`) provided for this session lacks a header row. In real-world banking data exports, particularly from legacy core banking systems (like T24 or Flexcube), headers are frequently omitted or malformed. We must manually assign logical column names based on standard Kenyan banking/SACCO churn schemas.

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from scipy import stats
6 from sklearn.model_selection import KFold
7 from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler,
   OrdinalEncoder
8 import warnings
9 warnings.filterwarnings('ignore')
10 sns.set_theme(style="whitegrid")
11
12 columns = [
13     'CustomerID', 'Age', 'Gender', 'MaritalStatus', 'Education',
14     'EmploymentStatus', 'Tenure', 'AccountType', 'NumProducts', 'Balance',
15     'SaccoScore', 'MonthsInactive', 'BranchVisits', 'IsStudent', '
   UtilizationRate',
16     'HasCrCard', 'HasLoan', 'IsMobile', 'IsInternet', 'Churn'
17 ]
18
19 df = pd.read_csv('churn_data.csv', header=None, names=columns)
20 df.head()

```

Listing 1: Assigning column names and loading the data

2.2 Inspecting Data Types and Memory

Before applying any transformations, we must verify that pandas has correctly inferred the data types. Incorrect data types are the root cause of silent failures in downstream modeling.

```
df.info()
```

Listing 2: Inspecting DataFrame structure and memory usage

Output Analysis:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            10000 non-null  object
1   Age                   10000 non-null  int64
2   Gender                10000 non-null  object
...

```

9	Balance	10000	non-null	int64
14	UtilizationRate	10000	non-null	float64
19	Churn	10000	non-null	int64

The Integer Identifier Trap

Notice that `CustomerID` is correctly loaded as an `object` (string). However, if a column like `BranchID` or `AccountNumber` were loaded as `int64`, distance-based algorithms (like KNN or K-Means) would calculate geometric distances between account numbers, which is mathematically nonsensical. Always cast identifier columns to `string` or `category` immediately upon loading.

Chapter 2 – Profiling Check

1. Calculate the percentage of missing values for every column in the dataset. Are there any columns that should be dropped entirely due to excessive missingness?
2. Identify all numerical columns and calculate their skewness using `df.skew()`. Which variable exhibits the most severe right-skew?
3. Count the number of unique categories in the `EmploymentStatus` and `Education` columns. How will this cardinality affect your encoding strategy?

3 Outlier Detection: Visual & Statistical Methods

An outlier is an observation that lies an abnormal distance from other values in a random sample from a population. In financial data, outliers are not merely statistical anomalies; they are often the most valuable customers in the portfolio. Therefore, detection must be rigorous, and treatment must be deeply considered.

3.1 Visual Detection: The Boxplot

The boxplot (or box-and-whisker plot) is the primary visual tool for identifying skewness and potential outliers in continuous variables.

```
1 plt.figure(figsize=(12, 4))
2 plt.subplot(1, 2, 1)
3 sns.boxplot(x=df['Balance'], color='lightblue')
4 plt.title('Boxplot: Account Balance (Highly Skewed)')
5
6 plt.subplot(1, 2, 2)
7 sns.boxplot(x=df['Age'], color='lightgreen')
8 plt.title('Boxplot: Customer Age')
9 plt.tight_layout()
10 plt.show()
```

Listing 3: Generating comparative boxplots for Balance and Age

Anatomy of the Boxplot:

- **The Box:** Represents the Interquartile Range (IQR), encompassing the middle 50% of the data (from the 25th percentile, Q_1 , to the 75th percentile, Q_3).
- **The Median Line:** The vertical line inside the box represents the 50th percentile (Q_2). In highly skewed data, the median is pushed toward the lower end of the box.
- **The Whiskers:** Extend to the furthest data points that fall within $1.5 \times \text{IQR}$ from the hinges (Q_1 and Q_3).
- **The Dots:** Any observation falling beyond the whiskers is plotted as an individual dot and flagged as a statistical outlier.

Visual Observation: The **Balance** boxplot shows a massive right tail with hundreds of dots stretching far beyond the upper whisker. The **Age** boxplot is relatively compact, with only a few isolated dots.

3.2 Statistical Detection: The IQR Method (Tukey's Fences)

The Interquartile Range (IQR) method, formalized by John Tukey, is a robust, non-parametric way to define outlier boundaries. Because it relies on quartiles rather than the mean, it is highly resistant to the influence of the extreme values it seeks to detect.

Tukey's Fences for Outlier Detection

Let Q_1 be the 25th percentile and Q_3 be the 75th percentile.

$$\text{IQR} = Q_3 - Q_1 \quad (1)$$

The boundaries for non-outlier data are defined as:

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR} \quad (2)$$

$$\text{Upper Bound} = Q_3 + 1.5 \times \text{IQR} \quad (3)$$

Any observation X where $X < \text{Lower Bound}$ or $X > \text{Upper Bound}$ is classified as an outlier. (Note: Using $3.0 \times \text{IQR}$ identifies “far out” or extreme outliers).

```
1 Q1 = df['Balance'].quantile(0.25)
2 Q3 = df['Balance'].quantile(0.75)
3 IQR = Q3 - Q1
4
5 lower_bound = Q1 - 1.5 * IQR
6 upper_bound = Q3 + 1.5 * IQR
7
8 outliers_iqr = df[(df['Balance'] < lower_bound) | (df['Balance'] > upper_bound)]
9 print(f"IQR Method: Found {len(outliers_iqr)} outliers (outside {lower_bound:.0f}
    } to {upper_bound:.0f}).")
```

Listing 4: Implementing the IQR Method on Account Balance

Output:

IQR Method: Found 503 outliers (outside -199286 to 450070).

3.3 Statistical Detection: The Z-Score Method

The Z-score method measures how many standard deviations a data point is from the mean. It relies heavily on the assumption that the underlying data follows a Gaussian (Normal) distribution.

The Z-Score and the Empirical Rule

The Z-score for an observation X is calculated as:

$$Z = \frac{X - \mu}{\sigma} \quad (4)$$

where μ is the population mean and σ is the standard deviation. Under the Empirical Rule for normal distributions, 99.7% of data falls within ± 3 standard deviations. Therefore, $|Z| > 3$ is the standard threshold for outlier classification.

```
1 z_scores = np.abs(stats.zscore(df['Balance'].dropna()))
2 outliers_z = df['Balance'][z_scores > 3]
3 print(f"Z-Score Method: Found {len(outliers_z)} outliers (Z > 3).")
```

Listing 5: Implementing the Z-Score Method on Account Balance

Output:

Z-Score Method: Found 177 outliers (Z > 3).

3.4 The Discrepancy: Why IQR and Z-Score Disagree

Notice the stark difference in results: IQR flagged **503** outliers, while Z-Score flagged only **177**. Why?

The Failure of Z-Score on Skewed Data

The Z-score method assumes a symmetrical, bell-shaped distribution. In highly right-skewed financial data, the extreme values in the tail mathematically pull the mean (μ) upward and massively inflate the standard deviation (σ). Because σ is artificially enlarged, the threshold for what constitutes "3 standard deviations away" becomes incredibly wide. Consequently, many extreme values "hide" within the inflated threshold and evade detection. **For skewed financial variables, the IQR method is mathematically superior.**

4 Outlier Treatment Decisions: The Business Context

Detecting an outlier is a statistical exercise; treating it is a business decision. The most common and fatal mistake made by junior data scientists is the automatic deletion of any row flagged as an outlier.

4.1 The Golden Rule: Outlier $\neq$ Error

An outlier is simply a value that lies far from the center of the distribution. An error is a value that is logically impossible or the result of a systemic failure.

The Outlier Decision Framework

Before removing an extreme observation, ask: *“Is this value impossible, or is it simply unusual?”*

1. **Data-entry error:** (e.g., Age = 150, Balance = -500). **Action:** Correct if possible, otherwise remove.
2. **Impossible value:** (e.g., Tenure > Customer Lifetime). **Action:** Investigate logic, correct or remove.
3. **Valid but extreme value:** (e.g., Balance = 2,000,000 KES). **Action:** Retain, but apply capping or transformation.
4. **Business-critical extreme:** (e.g., Institutional SME account). **Action:** Retain and carefully assess its influence; consider separate modeling.

High-Net-Worth Individuals and SMEs

In a Kenyan SACCO, a member with a savings balance of 5,000,000 KES is likely a high-net-worth individual, a successful SME, or an institutional investor (like a local chama or church group). If you delete these 503 "outliers" identified by the IQR method, you are systematically removing your most profitable, valuable customers from the dataset. This introduces severe **selection bias**, causing your churn prediction model to become entirely blind to the behaviors of the VIP segment.

4.2 Treatment Strategy: Capping (Winsorization)

Instead of deleting valid extreme observations, we limit their mathematical influence by replacing values above a certain threshold with the threshold value itself. This technique is known as **Winsorization** or **Capping**.

```
1 # Cap Balance
2 cap_val_balance = df['Balance'].quantile(0.95)
3 df['Balance_Capped'] = df['Balance'].clip(upper=cap_val_balance)
4
5 # Cap Age
6 cap_val_age = df['Age'].quantile(0.95)
7 df['Age_Capped'] = df['Age'].clip(upper=cap_val_age)
```

Listing 6: Capping Balance and Age at the 95th Percentile

The Mechanics of Capping: If the 95th percentile of Account Balance is calculated as 500,000 KES, any customer with a balance of 1,200,000 KES or 3,000,000 KES will have their analysis value set to exactly 500,000 KES.

Why the 95th Percentile?

The 95th percentile is a practical, business-friendly choice. It compresses the top 5% of extremes while leaving the bottom 95% of the distribution completely untouched. Note that this is not a universal statistical law; the threshold (90th, 95th, 99th) should be justified by the specific business context and the severity of the skew.

4.3 Advantages of Capping over Deletion

1. **Preserves Sample Size:** No customers are removed from the dataset. You retain all 10,000 records, maintaining maximum statistical power.
2. **Reduces Disproportionate Influence:** Distance-based algorithms (KNN, SVM, K-Means) calculate geometric distances. A balance of 5,000,000 would mathematically overpower an age of 40. Capping neutralizes this geometric distortion.
3. **Maintains Segment Representation:** High-net-worth individuals remain in the dataset, allowing the model to learn the churn behaviors of the SME/institutional segment, rather than ignoring them entirely.

```
1 plt.figure(figsize=(12, 4))
2 plt.subplot(1, 2, 1)
3 sns.boxplot(x=df['Balance_Capped'], color='lightblue')
4 plt.title('Boxplot: Account Balance (Capped)')
5
6 plt.subplot(1, 2, 2)
7 sns.boxplot(x=df['Age_Capped'], color='lightgreen')
8 plt.title('Boxplot: Customer Age (Capped)')
9 plt.tight_layout()
10 plt.show()
```

Listing 7: Visual confirmation of capping via Boxplots

Observation: The extreme dots have disappeared. The upper whisker now cleanly represents the compressed boundary of the data, making it “model-safe” for distance-sensitive algorithms.

5 Feature Encoding Strategies

Machine learning algorithms require numerical matrices. They cannot natively process text strings like “Male”, “Nairobi”, or “University”. However, naively mapping text to arbitrary numbers (e.g., Nairobi=1, Mombasa=2) injects false mathematical relationships into the model. We must choose an encoding strategy that respects the **cardinality** and **hierarchy** of the variable.

5.1 Strategy 1: Ordinal Encoding (Preserving Hierarchy)

Ordinal encoding is used when categories have a meaningful, natural order or progression.

```
1 edu_map = {'None': 0, 'Primary': 1, 'Secondary': 2, 'University': 3}
2 df['Education_Ord'] = df['Education'].map(edu_map)
3 print("Ordinal Encoding applied to 'Education'.")
```

Listing 8: Ordinal Encoding for Education Level

Why it works: The model correctly interprets that a University degree (3) represents a higher level of educational attainment than Primary school (1). The numerical distance carries semantic meaning.

The Ordinal Trap

Do *not* use Ordinal Encoding for nominal variables like “County” or “AccountType”. Assigning 1, 2, 3 to counties implies that County 3 is “three times greater” than County 1, which is mathematically nonsensical and will severely bias linear models.

5.2 Strategy 2: One-Hot Encoding (Low Cardinality)

One-Hot Encoding is used for nominal variables (no natural order) that have a **small number of unique categories**.

```
1 df = pd.get_dummies(df, columns=['Gender', 'MaritalStatus'],
2                          drop_first=True,
3                          dtype=int)
```

Listing 9: One-Hot Encoding for Gender and Marital Status

The Mechanics: Creates a new binary (0/1) dummy column for each category.

The Dummy Variable Trap

If a variable has k categories, One-Hot Encoding creates k binary columns. However, these columns are perfectly collinear (if you know the value of $k - 1$ columns, the k -th column is mathematically determined). This perfect multi-collinearity breaks the matrix inversion required by Logistic Regression and Linear Regression. **Solution:** Always use `drop_first=True` to drop one reference category, creating $k - 1$ linearly independent features.

5.3 Strategy 3: Target Encoding (High Cardinality)

Target Encoding is the professional solution for nominal variables with **many unique categories** (e.g., 47 Counties, 100 Branch IDs, or diverse Employment Statuses). Instead of creating dozens

of sparse dummy columns, we replace the text category with a numerical value based on its historical relationship with the **target variable**.

Example: If we encode “County” by its historical churn rate:

- Nairobi → 0.32 (High churn)
- Mombasa → 0.27
- Kisumu → 0.21 (Low churn)

A customer from Nairobi receives the dense, highly informative value of 0.32, reducing 46 dummy columns down to **one single feature**.

5.3.1 The Danger of Target Leakage

If you calculate the mean churn rate for Nairobi using the *entire* dataset, you are including the target value of the specific customer you are currently trying to predict. The model memorizes the exact churn rates of the training data, achieving 99% accuracy in training, but failing catastrophically on unseen test data. This is **Target Leakage**.

5.3.2 The Solution: K-Fold Target Encoding

To prevent leakage, we must calculate the target statistic using *out-of-sample* data. We achieve this through K-Fold Cross-Validation Target Encoding.

```
1 def kfold_target_encode(df, cat_col, target_col, n_splits=5):
2     """Educational K-Fold Target Encoder to strictly prevent data leakage."""
3     encoded_col = f"{cat_col}_TE"
4     df = df.copy()
5     df[encoded_col] = np.nan
6     global_mean = df[target_col].mean()
7
8     kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)
9     for train_idx, val_idx in kf.split(df):
10         # Calculate means ONLY on the training folds
11         fold_means = df.iloc[train_idx].groupby(cat_col)[target_col].mean()
12         # Map to the validation fold
13         mapped = df.iloc[val_idx][cat_col].map(fold_means).fillna(global_mean)
14         df.loc[df.index[val_idx], encoded_col] = mapped
15
16     return df
17
18 df = kfold_target_encode(df, 'EmploymentStatus', 'Churn')
19 print("Successfully encoded without leaking test-set information.")
```

Listing 10: Educational K-Fold Target Encoder to prevent leakage

The K-Fold Mechanics

1. Split the training data into K folds (e.g., $K = 5$).
2. For Fold 1, calculate the mean churn rate of each category using **only Folds 2, 3, 4, and 5**.
3. Apply these calculated means to encode the categories in Fold 1.

4. Repeat this process for all K folds.

Every row is encoded using data from *other* rows. No row's target value is used to encode itself. This strictly prevents target leakage while reducing dimensionality.

6 Scaling, Normalization & Algorithm Sensitivity

Scaling is the process of transforming numerical features so that they exist on a similar scale. However, scaling is not a universal requirement; it is an algorithm-specific intervention.

6.1 Why Scale Features?

Financial balances range from 0 to 1,000,000+. Customer ages range from 18 to 80.

- **Distance-Sensitive Models:** Algorithms like KNN, SVM, K-Means, and Neural Networks rely on geometric distance (e.g., Euclidean distance). A difference of 10,000 KES in balance will mathematically overpower a difference of 10 years in age. The model will effectively ignore the age variable.
- **Gradient-Descent Models:** Logistic Regression and Neural Networks converge much faster and more stably when features are on similar scales.
- **Tree-Based Models (The Exception):** Decision Trees, Random Forests, and XGBoost do *not* require scaling. They make decisions using orthogonal thresholds (e.g., “Is Balance > 100,000?”). They do not calculate distances between points. Scaling for tree models wastes computation.

6.2 Comparing Scalers: Standard vs MinMax vs Robust

```
1 scalars = {  
2     'StandardScaler': StandardScaler(),  
3     'MinMaxScaler': MinMaxScaler(),  
4     'RobustScaler': RobustScaler()  
5 }  
6  
7 for name, scaler in scalars.items():  
8     df[f'Balance_{name}'] = scaler.fit_transform(df[['Balance_Capped']])  
9     print(f"- {name}: Mean={df[f'Balance_{name}'].mean():.2f}, Std={df[f'  
Balance_{name}'].std():.2f}")
```

Listing 11: Applying and comparing three different scalers

Output:

- StandardScaler: Mean=-0.00, Std=1.00
- MinMaxScaler: Mean=0.31, Std=0.28
- RobustScaler: Mean=0.24, Std=0.77

Mathematical Formulations of Scalers

1. StandardScaler (Z-Score Normalization):

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

Centers data to Mean=0, Std=1. **Highly sensitive** to outliers.

2. MinMaxScaler:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Scales data to a fixed range, usually [0, 1]. **Extremely sensitive** to outliers.

3. RobustScaler:

$$X_{scaled} = \frac{X - \text{Median}}{\text{IQR}}$$

Centers to Median, scales using IQR. **Highly robust** to outliers and skew.

6.3 The MinMax Trap and The Robust Solution

If a single customer has a balance of 50M KES, **MinMaxScaler** will squash the other 9,999 customers into a microscopic range of 0.000 to 0.020, destroying their variance and rendering the feature useless to the model.

Even after capping at the 95th percentile, financial data retains skewness. **RobustScaler** uses the exact same logic as our outlier detection (Median and IQR). It ensures the bulk of the customers (the middle 50%) are spread out meaningfully, without being distorted by the capped extremes. It is the undisputed champion for skewed financial data.

6.4 Statistical Clarification: The Normality Myth

Dispelling the Normality Myth

Many analysts believe they must apply **StandardScaler** to force data into a normal (Gaussian) bell curve, or that Logistic Regression requires normally distributed predictors. **This is statistically false.**

- **StandardScaler does NOT make data normal.** It only shifts the mean to 0 and scales the variance. A skewed distribution remains skewed after StandardScaling.
- **Logistic Regression does NOT assume normal predictors.** It assumes a linear relationship between the log-odds of the target and the predictors.

We scale for **numerical stability**, comparability of coefficients, and optimization speed, *not* to satisfy a false normality assumption. If an algorithm genuinely requires normally distributed features (e.g., Linear Discriminant Analysis), you must use mathematical transformations like the **Log Transform** or **Box-Cox Transform**, not a scaler.

7 Practical Application & Final Pipeline

The final step of the preprocessing phase is to assemble the cleaned, encoded, and scaled features into a single, model-ready matrix, and to document the decisions made.

7.1 Selecting the Final Features

We filter the DataFrame to retain only the columns that have been properly treated and are ready for machine learning.

```
1 final_cols = [  
2     'Age', 'Tenure', 'NumProducts', 'SaccoScore', 'MonthsInactive', '  
3     'BranchVisits',  
4     'UtilizationRate', 'Education_Ord', 'EmploymentStatus_TE', '  
5     'Balance_RobustScaler',  
6     'Gender_Male', 'MaritalStatus_Married', 'MaritalStatus_Single', 'Churn'  
7 ]  
8  
9 # Filter only existing columns to prevent KeyErrors  
10 final_cols = [c for c in final_cols if c in df.columns]  
11 df_final = df[final_cols].dropna()  
12  
13 output_file = 'churn_transformed_session2.csv'  
14 df_final.to_csv(output_file, index=False)  
15 print(f"[SUCCESS] Transformed dataset saved as '{output_file}'")
```

Listing 12: Selecting final clean columns for the ML pipeline

7.2 The Preprocessing Memo (Deliverable)

A professional data scientist does not just write code; they write defensible documentation. Your deliverable for this session is a technical memo justifying your preprocessing pipeline.

Final Deliverable: The Preprocessing Memo

Draft a 1-page technical memo addressed to the Head of Data Science at a Kenyan Retail Bank. Your memo must include the following sections:

1. **Outlier Strategy:** Justify why Winsorization (capping at the 95th percentile) was chosen over Listwise Deletion for the **Balance** variable, specifically referencing the SAC-CO/SME business context.
2. **Encoding Strategy:** Explain why K-Fold Target Encoding was utilized for **EmploymentStatus** (and potentially **County**), detailing how the K-Fold mechanism mathematically prevents target leakage compared to naive mean-encoding.
3. **Scaling Strategy:** Justify the selection of **RobustScaler** over **StandardScaler** and **MinMaxScaler** for financial features, referencing the underlying mathematical formulas (Median/IQR vs Mean/Std).
4. **Algorithm Alignment:** Explicitly state which downstream models (e.g., KNN, Logistic Regression) require this scaling, and which models (e.g., XGBoost, Random Forest) would render this scaling step computationally redundant.

8 Best Practices Checklist

Use this checklist to validate your preprocessing pipeline before passing data to any machine learning estimator:

mclightStage	Action	Rationale
Outlier Detection	Use IQR for skewed data, Z-score for normal data.	IQR is robust to the long tails common in finance and demographics.
Outlier Treatment	Cap at 95th/99th percentile (Winsorization).	Retains valuable high-net-worth customers; prevents distance distortion.
Low-Cardinality	One-Hot Encoding (drop_first=True).	Prevents false ordinal relationships and the dummy variable trap.
High-Cardinality	K-Fold Target Encoding.	Reduces dimensions while strictly preventing target leakage.
Ordinal Hierarchy	Ordinal Mapping (e.g., Education).	Preserves the natural semantic progression of the categories.
Scaling	RobustScaler for skewed features.	Uses Median/IQR, ignoring extreme capped values.
Tree Models	Skip scaling entirely.	Trees use thresholds, not distances; scaling is wasted compute.

Table 1: The Professional Preprocessing Checklist.

The Ultimate Mindset

Understand the data → Identify the problem → Select the transformation → Justify the decision → Validate the result.

The objective is not merely to make the dataset “clean.” The objective is to produce reliable, representative, and model-ready data without unnecessarily destroying useful business information.

9 Glossary of Key Terms

Winsorization	The process of capping extreme values at a specified percentile (e.g., 95th) to limit their influence without deleting the observation.
IQR (Interquartile Range)	The difference between the 75th and 25th percentiles. A robust measure of statistical dispersion.
Target Leakage	A critical error where information from the target variable or test set inadvertently influences the training features, leading to overoptimistic evaluation.
K-Fold Encoding	A target encoding technique that calculates category statistics using out-of-fold data to strictly prevent target leakage.
Dummy Variable Trap	A state of perfect multi-collinearity caused by retaining all k dummy variables for a k -category nominal feature. Solved by dropping one reference category.
RobustScaler	A scaling algorithm that uses the median and IQR, making it highly resistant to outliers and skewness.
Cardinality	The number of unique categories within a categorical variable. High cardinality requires specialized encoding (e.g., Target Encoding).
Curse of Dimensionality	The phenomenon where adding too many sparse features (e.g., via One-Hot Encoding on high-cardinality data) degrades model performance and increases overfitting.
Selection Bias	Bias introduced when the data sample is not representative of the population, often caused by naively deleting outliers or missing rows.

10 References & Further Reading

- Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.
- Micci-Barreca, D. (2001). *A Preprocessing Scheme for High-Cardinality Categorical Attributes in Classification and Prediction Problems*. ACM SIGKDD Explorations Newsletter.
- scikit-learn Documentation: `RobustScaler`, `OrdinalEncoder`, and `KFold`.
- Mediacrest Training College — *Advanced Data Analytics & Machine Learning* curriculum (Weeks 1–8).

11 Session Deliverables & What Comes Next

Today's Deliverable — due end of session

- Completed **Preprocessing Memo** justifying outlier, encoding, and scaling decisions.
 - The final, model-ready dataset exported as `churn_transformed_session2.csv`.
 - Execution of the K-Fold Target Encoder function without triggering data leakage.
-
- **Day 2:** Feature Selection techniques (Filter, Wrapper, Embedded) and handling multi-collinearity (VIF).
 - **Day 3:** Dimensionality Reduction (PCA) and building the first predictive baselines (Logistic Regression vs. Random Forest).

Mediacrest Training College – Nairobi, Kenya
Advanced Data Analytics & Machine Learning · Week 2, Day 1
Trainer: Victor Wandera Lumumba · 18 August 2026
